

Operating Systems

EDA093 / DIT401

Lab 1 – Custom Shell

Laboratory Report

Group: Group 37

Authors: Malin Alcén
Felix Baur
Måns Bremer

Date: September 24, 2026

Contents

1	Introduction	2
1.1	Specifications	2
2	Theory	2
2.1	Processes	3
2.2	Process Creation and Program Execution	3
2.3	Foreground and Background Processes	3
2.4	Zombie Processes	3
2.5	Signals	3
2.6	Process Groups	4
2.7	File Descriptors and I/O Redirection	4
2.8	Pipes	4
3	Method	4
3.1	Running Basic Commands	5
3.2	Handling <code>Ctrl-D</code> and <code>exit</code>	5
3.3	Handling <code>Ctrl-C</code>	5
3.4	Background Execution	5
3.5	Eliminating Zombie Processes	6
3.6	I/O Redirection Implementation	6
3.7	Pipeline Implementation	7
3.8	Providing the <code>cd</code> Function	7
4	Results	8
4.1	Lab Test Suite	8
4.2	Lab Instructions	8
5	Discussion	8
5.1	Pipeline Process Structure	8
A	Code	9

1 Introduction

A shell provides an interface between the user and the operating system by interpreting commands and starting the corresponding programs. When a command is entered, the shell is responsible for creating and managing the processes required to execute it. For example, executing a simple command such as `ls` involves creating a child process in which the requested program is executed, while the parent shell manages the execution and waits for the process to terminate when appropriate.

The purpose of this laboratory was to develop a simple Unix shell, `lsh`, and thereby gain practical experience with process management and process communication in a Unix environment. The shell was required to support execution of standard commands, background processes, pipelines, input and output redirection, and the built-in commands `cd` and `exit`. It also needed to correctly handle signals such as `Ctrl-C` and `Ctrl-D`, while ensuring that terminated child processes were properly collected to prevent zombie processes.

There was skeleton code provided, which helped in the parsing of various commands, meaning that such implementation would not need to be created or explored by us.

The final result will be judged on whether or not the code meets the specifications outlined in subsection 1.1 as well as a self test provided by the course.

1.1 Specifications

The implemented shell, `lsh`, was required to satisfy the following specifications:

- Execute basic commands and locate programs using the `PATH` environment variable.
- Support background execution using `&`.
- Support one or more pipes between commands.
- Support input and output redirection.
- Provide `cd` and `exit` as built-in commands.
- Respond to `Ctrl-D` by exiting the shell.
- Respond to `Ctrl-C` such that foreground processes are terminated without terminating the shell itself, nor background processes. A background process must still react accordingly to a `SIGINT` signal.
- Ensure that no zombie processes remain.

Additionally, command execution had to be implemented directly without delegating execution to another system shell.

2 Theory

A number of new operating system and shell related concepts had to be explored in order to successfully complete the lab. This section briefly explains what these were and what they do.

2.1 Processes

A process is an instance of a program currently being executed by the operating system. The execution of a program may involve multiple processes working together. A process can also create one or more child processes, forming a hierarchy of parent and child processes. Each process has its own process ID and executes independently of other processes.

2.2 Process Creation and Program Execution

The `fork()` system call can be used by a process to create a new child process. The child initially receives a copy of the parent process and then continues execution independently. The parent and child can therefore perform different tasks after the fork. A parent process can use `wait()` or `waitpid()` with the right parameters to pause its execution until one of its child processes has terminated.

To execute a command, the child process can use one of the `exec()` functions. These replace the current process image with the executable associated with the requested command. In this lab, `execvp()` was used to locate and execute commands using the `PATH` environment variable.

2.3 Foreground and Background Processes

By default, commands entered in a shell are executed in the foreground. A command can instead be executed in the background by appending `&` to it. When a command runs in the foreground, the shell waits for it to finish before accepting another command. When a command runs in the background, the shell immediately returns control to the user, allowing additional commands to be entered while the background process continues executing.

2.4 Zombie Processes

A zombie process is a process that has finished executing but whose termination status has not yet been collected by its parent process. Until the parent calls a function such as `wait()` or `waitpid()`, the process remains in the process table as a zombie. Zombie processes are undesirable and generally indicates incorrect process management.

2.5 Signals

Signals are a mechanism used by operating systems to notify a process that a particular event has occurred. For example, when `Ctrl-C` is pressed, a process would normally receive the `SIGINT` signal. The behaviour of a process when receiving a specific signal can be changed from its default behaviour by using the `signal()` function, allowing the process to do things such as ignore the signal or execute a custom signal handler.

Apart from the `SIGINT` signal, the `SIGCHLD` signal played a vital role in this lab. The `SIGCHLD` signal is received by a parent process when the state of one of its child processes changes, most importantly when a child process terminates. This allows the parent process to react to the termination of a child and collect its exit status using functions such as `wait()` or `waitpid()`.

In the context of a shell, this is especially important for background processes, since the shell should still be usable whilst waiting for them to finish. By handling `SIGCHLD`, the shell can instead continue accepting new commands and later collect terminated background processes, thereby preventing them from remaining as zombie processes.

2.6 Process Groups

A process group is a collection of processes that can be treated as a single unit by the operating system. Processes normally inherit the process group of their parent, but this can be changed using functions such as `setpgid()`.

Process groups are important when handling terminal-generated signals. Pressing **Ctrl-C** causes the terminal to send **SIGINT** to its foreground process group. Background processes should therefore be placed in a separate process group so that they are not unintentionally terminated by a use of **Ctrl-C** intended for a foreground command. They may still use the default **SIGINT** behaviour and can therefore be terminated if a **SIGINT** signal is sent to them directly.

2.7 File Descriptors and I/O Redirection

In Unix-like operating systems, a file descriptor is an integer that uniquely identifies an open file or I/O stream for a process. By default, every process starts with three standard file descriptors: standard input (`stdin`), standard output (`stdout`), and standard error (`stderr`). When a command is executed, it typically reads from standard input and writes to standard output.

I/O redirection allows a user to change this default behavior by instructing the shell to read from or write to different files instead. For example, using the `>` operator redirects standard output to write to a file, while `<` redirects standard input to read from a file. In a C program, this is achieved by using the `open()` system call to acquire a file descriptor for the target file, and then utilizing the `dup2()` system call. The `dup2()` function duplicates the newly opened file descriptor onto the standard file descriptor, such as `STDIN_FILENO` or `STDOUT_FILENO`, effectively replacing it. Afterwards, the original file descriptor is closed using `close()` to prevent resource leaks. When the process subsequently calls an `exec()` function, the executed program automatically inherits these modified file descriptors, performing its I/O operations on the redirected files without needing to know about the change.

2.8 Pipes

A pipe is an inter-process communication mechanism that provides a unidirectional data channel between two processes. In a shell environment, pipes are commonly used to directly connect the standard output of one commands to the standard input of another one, without the need for temporary files.

This functionality is implemented using the `pipe()` system call, which creates a pipe and returns an array of two file descriptors. One designated for reading and one for writing. To establish a pipeline, a parent process creates a pipe and then uses `fork()` to spawn the communicating child processes. The writing child process uses the `dup2()` function to redirect its standard output to the write-end of the pipe. Similarly, the reading child process uses `dup2()` to redirect its standard input to the read-end of the pipe. Crucially, all processes must close any unused pipe file descriptors using `close()`. This prevents resource leaks and ensures that the reading process correctly receives an end-of-file (EOF) signal when the writer process has finished executing.

3 Method

This section describes our code's methods to meet the specifications outlined in subsection 1.1, using the knowledge explained in section 2.

3.1 Running Basic Commands

The handling of most basic shell commands, apart from the special cases described in the following subsections, was implemented by first creating a new child process using `fork()`. The child process then executed the requested command using `execvp()`, which also allowed the command to be located using the `PATH` environment variable. Meanwhile, the parent shell used `waitpid()` to wait for the specific child process to terminate before returning control to the user. A simplified version of this process can be seen in Listing 1.

Listing 1: Simplified execution of a basic foreground command.

```
1 pid_t pid = fork();
2
3 if (pid == 0) {
4     // Child process: execute the requested program
5     execvp(command, arguments);
6
7 } else if (pid > 0) {
8     // Parent process: wait for the child to terminate
9     waitpid(pid, NULL, 0);
10 }
```

3.2 Handling Ctrl-D and exit

The handling of `Ctrl-D` and typing `exit` in the shell were both implemented by terminating the main command-reading loop. The difference lies in how the two cases are detected.

When the user presses `Ctrl-D`, the terminal signals end-of-file (EOF) on the input stream. In this implementation, `readline()` then returns `NULL`. The shell checks for this condition and exits the loop if it occurs.

The `exit` command is handled by comparing the entered command string with the string `"exit"` after the whitespace has been removed. If they match, the shell also terminates the main loop.

In both cases, the program then reaches the end of `main()` and terminates normally.

3.3 Handling Ctrl-C

In order to handle `Ctrl-C` in the expected way, `signal(SIGINT, SIG_IGN)` is called before the shell's main loop to ensure that the main shell process ignores the `SIGINT` signal, as it would otherwise terminate when receiving it. When a new command is executed in the foreground, `signal(SIGINT, SIG_DFL)` is called within the child process to restore the default `SIGINT` behaviour for that process. This allows the foreground process to be terminated by `Ctrl-C` while the main shell remains running.

Background processes also restore the default `SIGINT` behaviour, so they could still be terminated if a `SIGINT` signal is sent to them directly. However, they are placed in a separate process group from the shell using `setpgid()`, preventing them from receiving the terminal-generated `SIGINT` caused by the user pressing `Ctrl-C`.

3.4 Background Execution

The skeleton code's parser already contained the capability of providing an integer indicating whether a command was intended to be run in the background or the foreground. An explanation of how commands running in the foreground are handled is found in subsection 3.1. The

handling of a background process starts deviating from the handling of a foreground process within the parent, after it has forked the child process. Instead of waiting for the child process to finish, the parent immediately returns to the main shell loop, allowing the user to enter new commands while the child process continues executing in the background.

The child process itself is executed in largely the same way as a foreground process, with the requested command being started using `execvp()`. The main difference is therefore how the parent process handles the child process after the fork. Since the parent process does not immediately call `waitpid()` for a background process, additional handling is required when the background process later terminates, in order to prevent it from remaining as a zombie process.

3.5 Eliminating Zombie Processes

As described in subsection 3.4, the shell does not immediately wait for background processes to terminate. Without additional handling, a background process that finishes execution would therefore remain as a zombie until its parent collects its termination status. To prevent terminated background processes from remaining as zombies, a custom handler for the `SIGCHLD` signal in the form of a very basic function was implemented.

The handler is installed in the main shell process before the main loop, using `signal(SIGCHLD, sigchld_handler())`. Whenever a child process terminates, the shell receives a `SIGCHLD` signal. This in turn causes the handler function to run, calling `waitpid()` with the `WNOHANG` option as seen in Listing 2. This allows the shell to collect terminated child processes without blocking the handling of other actions if no child is currently ready to be collected. Within the function a loop is used, because several child processes may have terminated before the signal handler is executed. Listing 2 shows what the function called every time the `SIGCHLD` signal is received looks like.

Listing 2: `SIGCHLD` handler function.

```
1 void sigchld_handler() {  
2     while (waitpid(-1, NULL, WNOHANG) > 0);  
3 }
```

When the shell executes a foreground command, the `SIGCHLD` handler is temporarily disabled while the parent process explicitly waits for that specific child process using `waitpid()`. After the foreground process has terminated, the custom handler is restored and an additional non-blocking `waitpid()` loop is performed to collect any other child processes that may have terminated during that time. This ensures that completed background processes are eventually removed from the process table.

3.6 I/O Redirection Implementation

The implementation of I/O redirection is handled by a custom `apply_redirection()` function, which is called by child processes before executing their designated commands using `execvp()`. The function takes two string arguments representing the file names for standard input and standard output redirection, which are provided by the parsed command structure.

If an input file is specified, the file is opened with read-only permission using the `open()` system call with the `O_RDONLY` flag. The `dup2()` system call is then used to duplicate the resulting file descriptor onto `STDIN_FILENO`. Afterwards, the original file descriptor is closed using `close()` to prevent resource leaks.

Similarly, if an output file is specified, it is opened using the `O_WRONLY | O_CREAT | O_TRUNC` flags. This ensures that the file is created with `0644` permissions if it does not already exist, or truncated if it does. `dup2()` is then used to overwrite `STDOUT_FILENO`. If any `open()` or `dup2()` calls fail, the child process prints an error message using `perror()` and safely terminates via `exit(1)`.

3.7 Pipeline Implementation

The pipeline functionality is implemented using a recursive function, `run_pipeline()`, which traverses the reverse-linked list of commands provided by the parser. To isolate the pipeline's execution from the main shell, an initial wrapper process is forked within the `run_piped()` function. This outer child process is responsible for setting up signals and initiating the recursive execution.

During each recursive step, a new pipe is created using the `pipe()` system call. The function then forks two child processes: a left child to handle the preceding commands in the pipeline, and a right child for the current command. The left child redirects its standard output to the write-end of the pipe using `dup2()` and recursively calls the function to handle the remainder of the command list. The right child redirects its standard input to the read-end of the pipe, applies any necessary file redirection, and executes its command using `execvp()`.

Crucially, both child processes and the parent wrapper close the original pipe file descriptors using `close()`. The parent wrapper then uses `waitpid()` to wait for both the left and right children to terminate. This ensures that the entire pipeline executes concurrently and completes successfully without leaving any zombie processes.

3.8 Providing the cd Function

A built-in `cd` function was provided by writing a custom one making use of the `chdir` function, as seen in Listing 3. The `chdir` function changes the current working directory to the directory given as argument when the `cd` function is called. If `arg` is `NULL`, meaning that the user has not specified a directory for the directory change, an error message will be printed.

Listing 3: Custom `cd` function.

```
1 void cd(char *arg) {  
2     if (arg == NULL) {  
3         fprintf(stderr, "Missing argument for cd \n");  
4         return;  
5     }  
6     if (chdir(arg) == -1) {  
7         perror("cd");  
8     }  
9 }
```

Since the `cd` command is executed differently than other commands, it also needed to be implemented differently. After the user has given their input, but before any command is executed, an `if`-statement is used to check if the user input is a `cd` command without any pipes. If it is, the `cd` function is executed immediately and the shell then returns to its main command-reading loop. If a pipeline is given instead, the `run_pipeline()` function explained in subsection 3.7 is called. Within the recursive structure of this function, `if`-statements are used to check for any `cd` commands. If a `cd` command is found, the `cd` function is executed immediately and the child process is then exited safely, letting the `run_pipeline()` function continue.

4 Results

This section presents two types of results. First, the implementation is evaluated using the automated test suite provided for the lab. Second, the final implementation is assessed against the specifications listed in subsection 1.1 and the additional behaviours described in the lab instructions.

4.1 Lab Test Suite

The final implementation was tested using the automated test suite provided for the lab and successfully passed 14 out of 15 tests. The reason for this test failing is detailed in subsection 5.1, and after speaking to the course teacher assistants we have received the green light for our code to be accepted as it is.

4.2 Lab Instructions

In addition to the automated test suite, the behaviours described in the lab README were tested manually. All of the specified cases behaved as expected. This included commands such as `cd lab_1 | abf`, where the non-existent `abf` command was expected to produce an error (while the shell itself continued operating correctly). A related case, `cd lab_1 | ls`, was also tested to verify that `cd` behaved correctly when used as part of a pipeline. These tests covered behaviours that were not necessarily included in the automated test suite.

5 Discussion

5.1 Pipeline Process Structure

Our pipeline implementation uses a recursive process creation strategy. For a command such as `sleep 30 | sleep 30 | sleep 30`, this results in the process structure shown in Figure 1. The additional processes are used to recursively handle the left side of the pipeline.

The three `sleep` processes are executed in parallel. At each recursive step, both child processes are created before the parent calls `waitpid()`. Therefore, the parent does not wait for one command to finish before creating the next one. The pipes only connect the processes and do not make their execution sequential.

This approach uses more resources than necessary. A flatter implementation could create the one wrapper and three command processes directly as four children of the shell. However, the recursive structure has the advantage of mapping naturally to an arbitrary number of pipeline commands. Each recursive step handles one pipe and creates the corresponding processes, making the implementation simple and general.

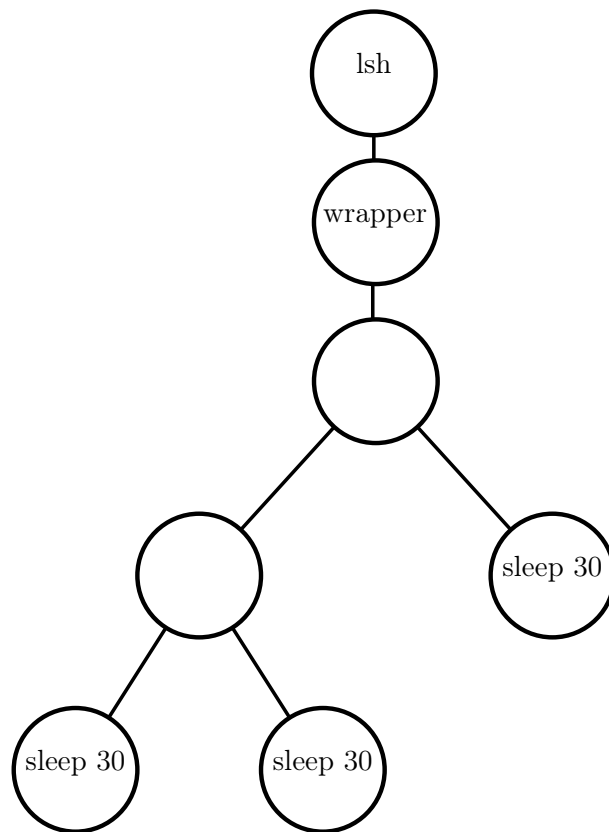


Figure 1: Process structure for `sleep 30 | sleep 30 | sleep 30`.

A Code

The code used throughout the lab can be found in our GitHub repository:
<https://github.com/Breman402/OperatingSystemsLabs>